

论文模板 06 · 系统集成与异构系统互操作 (gRPC / 契约驱动)

配套素材见 [00-项目素材库.md](#)。本主题是 BASE HOME 的绝对强项——「一份 proto 驱动 Java/Go/PHP 三语言」是极少有人能写出的真实素材，竞赛压力小，**强烈建议重点准备**。

一、真题常见问法

变体 A（系统集成整体）

请论述你参与的某系统的集成方案设计，包括集成方式选型、接口契约管理与异构系统互操作等方面。

变体 B（接口/契约管理）

请论述你如何在多团队、多技术栈环境下保证接口契约的一致性。

变体 C（企业应用集成 EAI）

请论述你如何实现企业内多个异构系统之间的数据集成与流程集成。

判断：变体 B 是本项目的**最优命中区**；变体 A、C 也可用同一套素材，只需调整论述重心。

二、摘要模板（约 300 字）

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台的系统集成架构设计与实现。该平台需同时为 Java 内部服务、Go 周边平台与 PHP 既有系统三类调用方提供服务，并与集团 MDM 主数据平台、外部预约系统、邮件与消息网关等多方交互。本人担任架构师，负责接口契约设计、异构系统集成方案与集成治理机制建设。针对【多技术栈下接口定义易漂移】、【依赖版本隐式解析引发运行时故障】、【外部系统故障影响业务连续性】三大挑战，本文从【契约单一来源】、【构建期版本治理】、【外部依赖本地化】三个维度进行设计：以 Protocol Buffers 作为唯一契约源，通过构建流水线生成 Java、Go、PHP 三语言产物及 OpenAPI 文档；以统一版本管理插件锁定关键依赖；以本地只读主数据副本隔离外部平台故障。系统上线后【 填效果】，验证了方案的可行性。

三、正文五段框架

第 1 段 · 项目背景与集成挑战 (约 400 字)

集成类论文的开头要写清「谁和谁集成」。

BASE HOME 处于集团 IT 版图的中位位置，集成关系复杂： - **上游**：集团 MDM 主数据平台（本国、城市、航空公司、货币等主数据） - **对外**：周边平台通过 Go 客户端接入，既有系统通过 PHP 客户端接入 - **对内**：Web 层通过 gRPC 调用 Service 层（17 个 RPC） - **下游**：邮件网关、SNS 消息推送、SQS 队列、DynamoDB Stream

多技术栈并存带来一个根本性问题：**接口定义在哪里是「真」的**？如果每个调用方各自维护一份接口定义，字段口径必然随时间漂移，而这类问题往往在联调甚至上线后才暴露。

第 2 段 · 架构设计 (约 800 字，本文核心)

第一层：契约单一来源 (Single Source of Truth)

我把 `base-home-spec` 模块确立为**唯一的接口契约源**，所有接口以 protobuf 定义，统一存放于 `his/base/home/v1/` 目录，按职责分层组织： - `basehome_service.proto` —— 服务定义（`BaseHomeService`，17 个 RPC） - `request/` —— 17 个请求消息 - `response/` —— 17 个响应消息 - `enums/` —— 13 个枚举（产品类型、文档类型、申请人类型、消息状态、销售渠道、发送者类型、保险类型等） - `type/` —— 约 42 个可复用消息（含统一的 `Error{code,message}`）

全部约 90 个 proto 文件纳入版本管理，任何接口变更都必须通过该模块提交，从流程上杜绝了「各写一份」的可能。

层次化组织的价值：`type/` 中的可复用消息（如地址、金额、旅行者基础信息）被多个 request/response 引用，避免了同一结构在 17 个接口中重复定义 17 次——这是契约可维护性的关键。

第二层：构建期多语言产物生成

契约定义统一后，我在构建脚本中编排了完整的生成链路：`build.sh` 按 `build-java` → `build-go` → `build-php` → `generate-all` 顺序执行，每种语言使用各自的官方插件：

目标语言	生成方式	产物形态
Java	<code>protobuf-gradle-plugin</code> (protoc + protoc-gen-grpc-java)	发布至内部 Maven
Go	容器内 protoc (<code>--go_out</code> / <code>--go-grpc_out</code> / <code>--grpc-gateway_out</code>)	Go module zip
PHP	<code>--php_out</code> + <code>grpc_php_plugin</code>	Composer 包
文档	<code>--openapiv2_out</code> → MkDocs	在线 API 文档

关键设计：把「生成」纳入构建流水线而非依赖人工执行，使接口一致性问题从「运行时联调发现」提前到「构建期暴露」——这是本次集成设计最重要的收益。

权衡说明：多语言生成的构建链路复杂度显著上升（需维护 Docker 化的 protoc 环境、三套发布流程），但相比接口不一致导致的线上缺陷与联调返工，这个成本是值得的。

第三层：对外依赖本地化

集团 MDM 主数据平台是外部系统，若业务查询直连外部，则外部系统的任何抖动都会直接传导到业务操作。我采用**本地只读副本**策略：通过 `ImportMdmJob` 定时从 MDM API 拉取主数据，落地为 26 张 `M*` 主数据表，业务侧只查本地。这样 MDM 平台短暂不可用时业务不受影响，数据新鲜度通过同步频率控制，是「可用性优先」的合理取舍。

第 3 段 • 关键技术实现 (约 700 字)

要素一：通信协议选型

服务间通信选择 **gRPC** 而非 REST，依据有三：① HTTP/2 多路复用，在高频内部调用下连接利用率更高；② protobuf 二进制编码体积小、序列化开销低；③ **强类型契约天然由 proto 保证**，与我们的契约单一来源策略一致。服务端监听 6565 端口，单次消息上限设为 64MB（旅游业务存在较大报文，如批量消息与文件列表），并开启 gRPC Reflection 便于调试期接口探查。

对外则通过 Web 层的 REST 接口（`/api/v1/**`）暴露，兼顾浏览器友好性与前端契约稳定性，并用 SpringDoc 生成 OpenAPI 描述（`/api-docs`）。

要素二：服务端的分发设计

`base-home-service` 以单一入口类 `BaseHomeService`（`@GRpcService`，继承 `proto` 生成的 `BaseHomeServiceImplBase`）承接 17 个 RPC，统一经 `execute(IGrpcService, req, respObserver)` 方法分发。业务逻辑委托给 18 个实现类（均实现 `IGrpcService` 接口）。这一设计的价值在于：**横切关注点集中**——日志、异常转换、认证上下文校验都可在 `execute` 中统一处理，各业务实现类只关注业务逻辑，避免每个 RPC 重复编写样板代码。

权衡说明：单一入口在 RPC 数量极大时会成为维护瓶颈，但 17 个 RPC 的规模下，统一分发带来的横切收益大于其复杂度成本。

要素三：跨业务域能力复用

多个 RPC 需要的能力（文档、消息线程、亲权同意书、序列号、旅行信息）下沉到 `service/shared/` 中的 5 个服务，避免同一逻辑在多个 RPC 中重复实现。文件相关能力单独归入 `service/filesystem/`。

要素四：错误契约统一

`proto` 中定义统一的 `Error{code, message}` 消息，所有接口的错误返回结构一致，调用方可编写统一的错误处理逻辑，无需为每个接口单独适配。错误码体系与 `his-common` 的消息 ID 规范对齐（如 `EABH00001` 形式的错误码），使错误在服务端、Web 端与文档中具有一致的标识。

第 4 段 • 难点与解决 (约 700 字)

集成主题最佳组合：

优先级	难点	适配理由
首选	① 一份 proto 契约驱动三语言	主题核心，独一无二
次选	② protobuf 版本冲突致启动失败	直接对应「集成治理」
三选	③ AWS SDK v1/v2 并存	直接对应「异构共存」
加分	⑩ MDM 主数据统一	直接对应「企业应用集成」

难点示例（②，三段式）：

问题： 契约统一后，新的问题出现了—— 契约的「消费者」不止我们的服务，还包括 protobuf 官方的公共类型库。一次依赖升级中， `proto-google-common-protos` 被解析到 2.64.0-rc1， gRPC 服务启动直接失败。更棘手的是，由于我们的版本声明采用了 动态区间 (`protobufVersion = '[3,4]'`、 `grpcVersion = 'latest.release'`)， 这个问题在构建期完全不显现，只在运行时暴露。

我的判断是：这不是偶发故障，而是**契约不完整**的表现—— 我们统一了业务接口的契约，却没有统一「第三方依赖版本」这一隐式契约。因此我没有做局部修复，而是在 `base-home-versions` 统一版本源中用 `resolutionStrategy { force ... }` 将 `proto-google-common-protos` 锁定到 2.63.2，并同步锁定 MyBatis 3.0.5 等关键组件。

效果： 启动失败问题消除；同一提交在任何机器上构建产物一致， **构建可复现性**得到保障。

第 5 段 • 总结与不足 (约 200 字)

目前该集成架构已稳定支撑 Java、Go、PHP 三端调用，
17 个 RPC 在三端口径一致，接口文档随契约自动生成。

不足之处：

一是跨语言契约虽来源统一，但**破坏性变更尚未实现自动化的兼容性检测**，
目前依赖人工评审；

二是 Go 与 PHP 侧的对齐仍依赖固定版本号人工同步，
尚未做到发布即对齐。

后续计划引入契约兼容性校验与三端发布联动。

通过本项目我认识到，系统集成的本质不是"把系统连起来"，
而是"让多个系统对同一件事保持一致的理解"。

四、本项目填充点速查

论文要素	直接可写的内容
契约源	<code>base-home-spec</code> ，protobuf 唯一契约源，约 90 个 proto 文件
服务定义	<code>basehome_service.proto</code> → <code>service BaseHomeService</code> ，17 个 RPC
契约分层	<code>request/</code> (17) + <code>response/</code> (17) + <code>enums/</code> (13) + <code>type/</code> (约 42)
统一错误	<code>type/error.proto</code> → <code>Error{code, message}</code>
多语言生成	Java (protobuf-gradle-plugin) / Go (protoc + grpc-gateway) / PHP (grpc_php_plugin)
生成编排	<code>build.sh</code> ：build-java → build-go → build-php → generate-all
文档生成	<code>--openapiv2_out</code> → <code>base_home_api.swagger.json</code> → MkDocs 站点
HTTP 映射	每个 RPC 带 <code>google.api.http</code> 映射，路径 <code>/v1/<RpcName></code>
服务通信	gRPC 端口 6565， <code>maxInboundMessageSize</code> 64MB，开启 Reflection
服务端设计	单入口 <code>BaseHomeService</code> + <code>execute()</code> 统一分发，18 个实现类

论文要素	直接可写的内容
能力复用	<code>service/shared/</code> 5 个共享服务; <code>service/filesystem/</code> 2 个
调用方式	配置化 endpoint (<code>tls://</code>) + TLS 传输加密
对外接口	Web 层 REST <code>/api/v1/**</code> , SpringDoc OpenAPI <code>/api-docs</code>
主数据集成	<code>ImportMdmJob</code> 定时从 MDM API 同步, 本地 26 张 <code>M*</code> 只读副本
外部预约集成	<code>ExternalReservationImportJob</code> + <code>RecvImportType</code>
异步集成	SNS 主题、SQS 队列、DynamoDB Stream → Lambda
版本治理	<code>base-home-versions</code> 插件 + <code>resolutionStrategy force</code>
版本对齐	<code>his-common-spec</code> 2.6.0 在 Java/Go/PHP 三侧对齐固定
依赖方向	web/service → common + spec; batch → common + mdm-spec
文档发布	MkDocs 站点 (容器化, 端口 8888)

五、可用难点素材（按适配度排序）

- 1. ① 一份 proto 契约驱动三语言 ★★★★★ —— 本主题核心
- 2. ② protobuf 版本冲突致启动失败 ★★★★★ —— 集成治理
- 3. ③ AWS SDK v1/v2 并存 ★★★★★ —— 异构共存与渐进演进
- 4. ⑩ MDM 主数据统一 ★★★★★ —— 企业应用集成
- 5. ⑥ Serverless 数据同步 ★★ —— 异步集成

六、踩坑提醒

坑	后果	正确做法
只说「用了 gRPC」	无深度	必须写 为什么不用 REST (HTTP/2、强类型契约、二进制编码效率)
不写契约管理	关键失分	「契约单一来源」是本主题的灵魂, 务必写透
编造服务注册中心	高风险	本项目 gRPC 为 endpoint 配置直连 , 无 Eureka/Nacos/Consul
编造 API 网关	高风险	本项目未见网关组件, 不要写「通过网关统一鉴权」; 可写「统一路径便于网关层集中治理」作为设计意图
编造 ESB	高风险	本项目未用 ESB, 集成靠契约 + 消息队列, 如实写即可
忽略版本治理	漏考点	集成不仅是「连起来」, 还包括依赖版本的一致性
忘记写权衡	缺乏架构思维	多语言生成的构建复杂度 vs 接口不一致的返工成本, 要明确对比

💡 **本主题的独特优势：**「一份契约生成 Java/Go/PHP 三语言 + 自动生成 OpenAPI 文档」这个素材**绝大多数考生写不出来**，因为它需要真实的多语言项目经历。如果你的考题涉及「接口管理」「多团队协作」「异构系统集成」，优先选这道题，用 ① 和 ② 两个难点撑满「难点」段，基本可以保证在这一题上拿到高于平均的分数。

配套：01-微服务架构设计.md（服务间通信）· 03-数据架构与混合存储.md（数据集成）